
Gestão de Hotéis

Projeto de Engenharia de Software em Java

Edição v0.1.0

Dr. Italo S. Vega



Gestão de Hotéis

Projeto de Engenharia de Software em Java

Edição v0.1.0



Esta versão pode ser consultada e baixada gratuitamente apenas para uso pessoal. Não deve ser redistribuída, vendida ou utilizada em trabalhos derivados.

© Dr. Italo S. Vega, 2026.

Sumário

1. Introdução	3
1.1. Visão Geral do Projeto	3
1.2. Papéis em um Processo de Desenvolvimento Típico	4
2. Desenvolvimento Orientado a Testes (TDD)	6
2.1. Evidência no Git	6
2.2. Onde os testes devem estar	6
2.3. Cobertura mínima	7
3. Requisitos Funcionais	8
4. Restrições Arquiteturais	9
5. Estruturas de Dados Obrigatórias	11
5.1. Status.java (Enum)	11
5.2. Hospede.java	11
5.3. Apartamento.java	12
5.4. Hotel.java	13
6. Módulo de Expansão: Hierarquia de Apartamentos	14
6.1. ApartamentoSimples.java	14
6.2. ApartamentoPremium.java	14
7. Módulo de Expansão: Serviços e Consumo	16
7.1. Servico.java	16
7.2. Consumo.java	16
8. Módulo de Expansão: Fatura	18
8.1. Fatura.java	18
9. Módulo de Expansão: Persistência	20
9.1. HotelRepo.java	20
10. Módulo de Expansão: Relatórios	21
10.1. RelatorioSvc.java	21
11. Contratos de Método	22

11.1. Comportamento Esperado para Cada Operação	23
12. Menu do Sistema	24
13. Validações Obrigatórias	26
14. Boas Práticas de Orientação a Objetos e Engenharia de Software .	27
15. Entregáveis	28
16. Critérios de Avaliação	29
16.1. CJ11 e CJ21: Avaliação TDD (demonstração em aula, 10 min por grupo)	29
16.2. CJ12A/B: Avaliação de Apresentação e Documentação Técnica (Operações + Hierarquia)	29
16.3. CJ22A/B: Avaliação de Apresentação e Documentação Técnica (Final) .	30
17. Calendário de Entregas	31
18. Dicas para Começar	33
19. Compilação e Execução	34
20. Observações Finais	35

1

Introdução

Você faz parte de uma equipe de até 5 colegas contratada para desenvolver um **Sistema de Controle de Reservas e Ocupação** para um hotel de médio porte¹. O sistema deverá gerenciar 20 andares com 14 apartamentos por andar, permitindo reservas, *check-ins*, *check-outs*, cancelamentos, consultas de ocupação, serviços de consumo, faturamento, persistência em arquivo e relatórios gerenciais.

Este projeto não é apenas sobre “fazer funcionar”. É sobre *aplicar os princípios da Orientação a Objetos* (encapsulamento, herança, responsabilidade única, baixo acoplamento) e entregar *software legível, manutenível e bem documentado*, simulando um ambiente de desenvolvimento profissional em equipe.

1.1. Visão Geral do Projeto

Item	Descrição
Linguagem	Java (versão 11 ou superior)
Paradigma	Orientação a Objetos (encapsulamento, herança, composição)
Equipe	Até 5 alunos, cada um responsável por um módulo específico
Estrutura	Pacote hotel com 13 classes/enums, distribuídas em módulos
Interação	Terminal (menu textual + Scanner) operado por funcionário da recepção
Compilação	<code>javac -d build hotel/**/*.java && java -cp build hotel.Main</code>

¹ Inspirado no projeto de Laboratório de Introdução à Computação (LIC) de 2026 proposto pelo Dr. Eli B. L. Costa.

1.2. Papéis em um Processo de Desenvolvimento Típico

Neste projeto, cada membro da equipe **atuará em todos os papéis**, típicos em um desenvolvimento real, de forma distribuída. Cada um será responsável por um ou mais módulos, mas todos devem compreender a integração entre eles.

Papel	Responsabilidade principal	Como a equipe exercerá este papel no projeto
Desenvolvedor	Escrever código que atenda aos requisitos funcionais, seguindo as boas práticas e padrões da linguagem.	Cada membro implementa as classes do seu módulo (Hospede, Apartamento, Servico, Fatura, etc.), garantindo que os métodos realizem corretamente as operações.
Arquiteto de Software	Definir a estrutura geral do sistema (quais classes, quais métodos, como elas se comunicam).	A equipe recebe uma arquitetura definida (assinaturas e divisão de responsabilidades). A tarefa é respeitá-la e entender por que ela foi proposta. Um líder de arquitetura pode ser designado.
Testador (QA)	Criar e executar testes para verificar se o software funciona corretamente e para descobrir bugs.	Cada membro escreve os testes antes da implementação (TDD, ciclo RED → GREEN → REFACTOR). Testes unitários para cada método público do seu módulo, mais testes de integração entre módulos (ex: Consumo COM Fatura).
Documentador	Escrever documentação técnica (Javadoc) e de usuário (menus, mensagens).	Cada membro documenta os métodos públicos do seu módulo. A documentação deve ser consistente em toda a base de código.
Gerente de Configuração	Controlar versões do código, organizar a estrutura de diretórios e garantir que a compilação seja reproduzível.	A equipe usa git com branching (cada módulo em uma branch) e faz merge para integração. Commits pequenos e frequentes.

1.2. Papéis em um Processo de Desenvolvimento Típico

Analista de Requisitos

Garantir que o software atende aos requisitos funcionais (REQ01 a REQ14).

Antes de cada entrega, a equipe verifica cada requisito com testes automatizados, assegurando que nada foi esquecido.

Suporte/DevOps

Garantir que o *software* compile e execute no ambiente-alvo (terminal Java).

A equipe testa a compilação e execução em ambiente limpo antes de cada entrega. A integração contínua é incentivada.

Reflexão importante: Um bom profissional de *software* não é apenas aquele que “coda rápido”. É aquele que consegue assumir diferentes perspectivas: testar o próprio código, documentá-lo, planejar sua estrutura e garantir que ele resolva o problema certo. Este projeto foi desenhado para que você pratique **todas** essas dimensões.

2

Desenvolvimento Orientado a Testes (TDD)

Todo o código deve ser desenvolvido seguindo o ciclo TDD (Test-Driven Development):

1. **RED** — Escreva um teste que falha para o método que vai implementar.
2. **GREEN** — Implemente o mínimo de código necessário para o teste passar.
3. **REFACTOR** — Melhore o código mantendo todos os testes verdes.

2.1. Evidência no Git

Os *commits* devem refletir o ciclo TDD:

- *Commits* com prefixo RED: contêm apenas a adição/alteração de testes (que falham se executados).
- *Commits* com prefixo GREEN: contêm a implementação que faz os testes passarem.
- *Commits* com prefixo REFACTOR: contêm melhorias sem mudança de comportamento.

O histórico de *commits* será verificado pelo professor como parte da avaliação. *Commits* que misturem teste + implementação no **mesmo** *commit* serão penalizados.

2.2. Onde os testes devem estar

Crie um arquivo `hotel/apresentacao/HotelTest.java` com todos os testes da equipe (unitários e de integração). Use asserções simples (`if` + `System.out`). Não é obrigatório usar JUnit. A estrutura recomendada:

```
package hotel.apresentacao;

public class HotelTest {
    private static int passou = 0;
    private static int total = 0;

    public static void main(String[] args) {
        testarReservarAptoLivre();
        testarReservarAptoOcupadoFalha();
        // ... todos os testes
        System.out.println(passou + "/" + total + " testes passaram");
    }

    static void testarReservarAptoLivre() {
        total++;
        Hotel hotel = new Hotel();
        Hospede h = new Hospede("123", "João", "Rua X", "9999", "joao@x");
        if (hotel.reservarApartamento(0, 0, h) && hotel.getApartamento(0,
0).estaReservado()) {
            passou++;
        } else {
            System.out.println("FALHOU: testarReservarAptoLivre");
        }
    }
}
```

2.3. Cobertura mínima

Cada método público de cada classe deve ter **pelo menos um teste** que verifique o comportamento esperado (caso feliz) e **pelo menos um teste** que verifique o comportamento em caso de erro/validação (caso triste).

3

Requisitos Funcionais

Nota sobre canais de interação: O cliente não interage diretamente com o sistema. A comunicação do cliente com o hotel (presencial, telefone, e-mail) é externa ao sistema. Quem opera o terminal é sempre um funcionário da recepção em nome do cliente.

Os requisitos abaixo (REQ01 a REQ14) são **obrigatórios** e servirão como base para a correção automatizada.

- REQ01** Visualizar mapa geral de ocupação (ex: . = livre, R = reservado, 0 = ocupado)
- REQ02** Reservar um apartamento (deve estar livre)
- REQ03** Realizar *check-in* a partir de uma reserva existente
- REQ04** Realizar *check-in* sem reserva (*check-in* direto em apartamento livre)
- REQ05** Realizar *check-out* (liberar apartamento ocupado)
- REQ06** Cancelar uma reserva (apenas se estiver reservada)
- REQ07** Visualizar taxas de ocupação e de reservas (%)
- REQ08** Cadastrar dados do hóspede no *check-in* (CPF, nome, endereço, celular, email)
- REQ09** Consultar situação de um apartamento específico + dados do hóspede (se ocupado ou reservado)
- REQ10** Cadastrar serviços (nome e preço) no cardápio do hotel e registrar consumo de serviços em apartamentos ocupados
- REQ11** Emitir a fatura de um apartamento, calculando valor total (diárias × preço da diária + consumos)
- REQ12** Diferenciar apartamentos por tipo (Simples ou Premium), com preços de diária distintos
- REQ13** Persistir e recuperar o estado do hotel em arquivo (salvar/carregar)
- REQ14** Gerar relatórios gerenciais: histórico de ocupação e *ranking* de serviços mais consumidos

4

Restrições Arquiteturais

O seu projeto **obrigatoriamente** deve seguir esta estrutura de pacotes e classes, dividida em **módulos** que podem ser desenvolvidos em paralelo por diferentes membros da equipe:

```
projeto_hotel/
├── hotel/
│   ├── apresentacao/
│   │   ├── Main.java           = Ponto de entrada e menu de interação
│   │   └── HotelTest.java      = Testes unitários e de integração
│   ├── negocio/
│   │   └── Hotel.java          = Fachada que gerencia a coleção de apartamentos
│   ├── modelo/
│   │   ├── Apartamento.java    = Classe base para apartamentos
│   │   └── ApartamentoPremium.java = Apartamento com diária premium (herda de
│   │       Apartamento)
│   │   ├── ApartamentoSimples.java = Apartamento com diária padrão (herda de
│   │       Apartamento)
│   │   └── Consumo.java        = Registro de consumo de serviço em um
│   │       apartamento
│   ├── Fatura.java             = Cálculo de fatura por apartamento
│   ├── Hospede.java            = Classe que armazena dados do hóspede
│   ├── Servico.java             = Item do cardápio do hotel (nome, preço)
│   └── Status.java              = Enum: LIVRE, RESERVADO, OCUPADO
│   ├── repositorio/
│   │   └── HotelRepo.java       = Persistência em arquivo do estado do hotel
│   └── servico/
│       └── RelatorioSvc.java     = Geração de relatórios gerenciais
```

Regras arquiteturais:

1. **Arquitetura em camadas:** O sistema segue uma arquitetura em camadas (Layered), com comunicação unidirecional: camadas superiores chamam as inferiores.

Camada

Classes

Chama

4. Restrições Arquiteturais

Apresentação	Main	Hotel, RelatorioSvc, HotelRepo
Negócio/Fachada	Hotel	Modelo
Serviço	RelatorioSvc	Modelo
Persistência	HotelRepo	Modelo
Modelo	Apartamento, Hospede, Servico, Consumo, Fatura, Status	Nenhuma — não chamam outras camadas

Note a diferença entre a **camada de modelo** e a **camada de negócio**:

- **Modelo** contém as entidades que representam os dados do domínio (Apartamento, Hospede, Fatura, etc.). Elas definem estado, construtores, validação local e `toString()` — mas não orquestram operações nem contêm lógica de coordenação.
- **Negócio** (Hotel) orquestra as operações sobre os modelos, coordena múltiplos objetos, aplica regras de invariante e expõe os casos de uso do sistema. É a fachada que a apresentação chama.

Regras fundamentais:

- Nenhuma camada chama a camada de apresentação (Main).
 - As classes de modelo **não** contêm `System.out.println` (exceto `toString()`, `mostrarMapa()` e `consultarApartamento()`).
2. **Encapsulamento**: Todos os atributos devem ser `private`. Acesso deve ser feito via *getters/setters* quando necessário.
 3. **Responsabilidade única**: Cada módulo tem uma responsabilidade clara. Hotel coordena as operações (padrão Fachada). Main apenas lida com entrada/saída e chama métodos do Hotel.
 4. **Herança**: `ApartamentoSimples` e `ApartamentoPremium` estendem `Apartamento`, aproveitando os métodos de transição de estado e adicionando `getPrecoDiaria()` (Template Method).
 5. **Composição**: `Consumo` associa `Servico` a um `Apartamento`. `Fatura` agrega consumos e calcula o total.
 6. **Nenhum `System.out.println` nas classes de modelo** (exceto `toString()` ou métodos explicitamente de exibição como `mostrarMapa` e `consultarApartamento`).
 7. **Tratamento de exceções**: Use exceções não checadas (`IllegalArgumentException`) para indicar erros de validação, mas **não capture exceções dentro das classes de modelo** (deixe para o Main).

5

Estruturas de Dados Obrigatórias

Você deve implementar exatamente estas classes e enum.

5.1. Status.java (Enum)

```
package hotel.modelo;

public enum Status {
    LIVRE,
    RESERVADO,
    OCUPADO;
}
```

5.2. Hospede.java

```
package hotel.modelo;

public class Hospede implements Serializable {
    private final String cpf;
    private final String nome;
    private final String endereco;
    private final String celular;
    private final String email;

    // Construtor com todos os campos e validação @NonNull
    public Hospede(String cpf, String nome, String endereco, String celular, String
email) { ... }

    // Getters (sem setters – objeto imutável)
    public String getCpf() { ... }
    public String getNome() { ... }
    public String getEndereco() { ... }
    public String getCelular() { ... }
    public String getEmail() { ... }
```

5. Estruturas de Dados Obrigatórias

```
@Override
public String toString() { ... }
@Override
public boolean equals(Object o) { ... } // baseado no CPF
@Override
public int hashCode() { ... } // baseado no CPF
}
```

5.3. Apartamento.java

```
package hotel.modelo;

import java.io.Serializable;

public class Apartamento implements Serializable {
    private Status status;
    private Hospede hospede; // null se status == LIVRE

    public Apartamento() {
        this.status = Status.LIVRE;
        this.hospede = null;
    }

    // Getters
    public Status getStatus() { ... }
    public Hospede getHospede() { ... }

    // Métodos de transição de estado (lançam IllegalStateException se transição
    inválida)
    public void reservar(Hospede h) { ... } // LIVRE -> RESERVADO
    public void checkin(Hospede h) { ... } // LIVRE/RESERVADO -> OCUPADO
    public void checkout() { ... } // OCUPADO -> LIVRE
    public void cancelarReserva() { ... } // RESERVADO -> LIVRE

    // Consultas de estado
    public boolean estaLivre() { ... }
    public boolean estaReservado() { ... }
    public boolean estaOcupado() { ... }

    // Preço da diária (0 na base; sobrescrito nas subclasses)
    public float getPrecoDiaria() { return 0f; }

    // Exibição formatada para o mapa
    public char getSymbol() {
        switch(status) {
            case LIVRE: return '.';
            case RESERVADO: return 'R';
            case OCUPADO: return 'O';
            default: return '?';
        }
    }
}

@Override
```

```
    public String toString() { ... }
}
```

5.4. Hotel.java

```
package hotel.negocio;

import java.io.Serializable;

public class Hotel implements Serializable {
    public static final int NUM_ANDARES = 20;
    public static final int APTOS_POR_ANDAR = 14;
    public static final int SIMPLES_POR_ANDAR = 8;

    private Apartamento[][] matriz;
    private List<Servico> servicos;
    private List<Consumo> consumos;

    public Hotel() {
        this.matriz = new Apartamento[NUM_ANDARES][APTOS_POR_ANDAR];
        this.servicos = new ArrayList<>();
        this.consumos = new ArrayList<>();
        inicializar();
    }

    // Inicializa: 8 ApartamentoSimples + 6 ApartamentoPremium por andar
    private void inicializar() { ... }

    // Validação de coordenadas
    private boolean aptoValido(int andar, int numero) { ... }
    private void validarCoordenadas(int andar, int numero) { ... } // lança
    IllegalArgumentException

    // Operações principais
    public boolean reservarApartamento(int andar, int numero, Hospede hospede)
    { ... }
    public boolean realizarCheckin(int andar, int numero, Hospede hospede) { ... }
    public boolean realizarCheckout(int andar, int numero) { ... }
    public boolean cancelarReserva(int andar, int numero) { ... }

    // Consultas
    public void mostrarMapa() { ... }
    public void consultarApartamento(int andar, int numero) { ... }
    public float calcularTaxaOcupacao() { ... }
    public float calcularTaxaReservas() { ... }

    // Getters controlados
    public Apartamento getApartamento(int andar, int numero) { ... } // valida
    coordenadas
    public List<Servico> getServicos() { ... }
    public List<Consumo> getConsumos() { ... }
    public List<Consumo> getConsumosDoApartamento(int andar, int numero) { ... }
    public boolean estaCompletoVazio() { ... } // true se todos LIVRE
}
```

6

Módulo de Expansão: Hierarquia de Apartamentos

6.1. ApartamentoSimples.java

```
package hotel.modelo;

public class ApartamentoSimples extends Apartamento {
    private static final float PRECO_DIARIA = 150.0f;

    public ApartamentoSimples() {
        super();
    }

    public float getPrecoDiaria() {
        return PRECO_DIARIA;
    }
}
```

6.2. ApartamentoPremium.java

```
package hotel.modelo;

public class ApartamentoPremium extends Apartamento {
    private static final float PRECO_DIARIA = 350.0f;

    public ApartamentoPremium() {
        super();
    }

    public float getPrecoDiaria() {
        return PRECO_DIARIA;
    }
}
```

```
    }  
}
```

A classe `Hotel.inicializar()` deve criar uma mistura de apartamentos Simples e Premium (ex: 8 Simples + 6 Premium por andar).

7

Módulo de Expansão: Serviços e Consumo

7.1. Servico.java

```
package hotel.modelo;

import java.io.Serializable;

public class Servico implements Serializable {
    private final String nome;
    private final float preco;

    public Servico(String nome, float preco) { ... }

    // Getters (sem setters – objeto imutável)
    public String getNome() { ... }
    public float getPreco() { ... }

    @Override
    public String toString() { ... }
}
```

7.2. Consumo.java

```
package hotel.modelo;

import java.io.Serializable;

public class Consumo implements Serializable {
    private final Apartamento apartamento;
    private final Servico servico;
    private final int quantidade;
}
```

```

public Consumo(Apartamento apto, Servico servico, int quantidade) { ... }

public Apartamento getApartamento() { ... }
public Servico getServico() { ... }
public int getQuantidade() { ... }

public float getTotal() {
    return servico.getPreco() * quantidade;
}

@Override
public String toString() { ... }
}

```

A classe Hotel deve conter uma lista de serviços cadastrados (List<Servico>) e uma lista de consumos registrados (List<Consumo>). Métodos adicionais em Hotel:

```

public void cadastrarServico(String nome, float preco);
public boolean registrarConsumo(int andar, int numero, int indiceServico, int
quantidade);
public List<Consumo> getConsumosDoApartamento(int andar, int numero);

```

8

Módulo de Expansão: Fatura

8.1. Fatura.java

```
package hotel.modelo;

import java.util.List;

public class Fatura {
    private Apartamento apartamento;
    private int dias;
    private List<Consumo> consumos;
    private float totalDiarias;
    private float totalConsumos;

    public Fatura(Apartamento apto, int dias, List<Consumo> consumos) {
        this.apartamento = apto;
        this.dias = dias;
        this.consumos = consumos;
        this.totalDiarias = calcularTotalDiarias();
        this.totalConsumos = calcularTotalConsumos();
    }

    private float calcularTotalDiarias() {
        return dias * apartamento.getPrecoDiaria();
    }

    private float calcularTotalConsumos() { ... }

    public float getTotalGeral() {
        return totalDiarias + totalConsumos;
    }

    // Getters
    public Apartamento getApartamento() { ... }
    public int getDias() { ... }
```

```
public float getTotalDiarias() { ... }  
public float getTotalConsumos() { ... }  
public List<Consumo> getConsumos() { ... }  
  
@Override  
public String toString() { ... }  
}
```

A classe Hotel deve ter um método para emitir fatura:

```
public Fatura emitirFatura(int andar, int numero, int dias);
```

Nota: Apartamento precisa de um método `getPrecoDiaria()`, definido nas subclasses `ApartamentoSimple`s e `ApartamentoPremium`. Se o apartamento for da classe base, retorne 0 ou lance exceção.

9

Módulo de Expansão: Persistência

9.1. HotelRepo.java

```
package hotel.repositorio;

import java.io.*;

public class HotelRepo {
    private static final String ARQUIVO = "hotel.dat";

    public static void salvar(Hotel hotel) throws IOException { ... }
    public static Hotel carregar() throws IOException, ClassNotFoundException { ... }
}
```

A classe `Hotel` deve implementar `java.io.Serializable` para que seus objetos possam ser serializados. O `Main` deve oferecer opções no menu para salvar e carregar.

10

Módulo de Expansão: Relatórios

10.1. RelatorioSvc.java

```
package hotel.servico;

import java.util.List;
import java.util.Map;

public class RelatorioSvc {
    private Hotel hotel;

    public RelatorioSvc(Hotel hotel) { ... }

    // Relatório 1: Histórico de ocupação (quantos dias cada apto ficou ocupado)
    public void gerarRelatorioOcupacao() { ... }

    // Relatório 2: Ranking de serviços mais consumidos
    public void gerarRankingServicos() { ... }

    // Relatório 3: Resumo financeiro (faturamento total estimado)
    public void gerarResumoFinanceiro() { ... }
}
```

11

Contratos de Método

Cada método público **deve** ser precedido por um comentário Javadoc no formato abaixo. Isso será cobrado na correção e na defesa oral.

Exemplo para o método `reservarApartamento` na classe `Hotel`:

```
/*
 * Reserva um apartamento, mudando seu status de LIVRE para RESERVADO.
 *
 * @param andar    Número do andar (0 a 19)
 * @param numero   Número do apartamento no andar (0 a 13)
 * @param hospede  Dados do hóspede que fará a reserva
 * @return true se a reserva foi bem-sucedida, false caso contrário
 * @throws IllegalArgumentException se andar ou número forem inválidos
 *
 * @pre O hotel deve estar inicializado (construtor já o faz)
 * @post Se bem-sucedido, o apartamento terá status RESERVADO e o hóspede será
armazenado
 */
public boolean reservarApartamento(int andar, int numero, Hospede hospede) {
    // validação
    if (!aptoValido(andar, numero)) {
        throw new IllegalArgumentException("Andar ou número inválido");
    }
    // lógica de negócio...
}
```

Você deve documentar TODOS os métodos públicos seguindo este padrão com as tags:

- `@param`
- `@return`
- `@throws`
- `@pre` (pré-condição)
- `@post` (pós-condição)

11.1. Comportamento Esperado para Cada Operação

Método	Comportamento resumido	Retorna true se...	Lança exceção se...
Hotel() (construtor)	Inicializa hotel com todos apartamentos LIVRE.	(void)	(nunca)
reservarApartamento	Se LIVRE → muda para RESERVADO e armazena hospede.	reserva bem-sucedida	andar/apto inválido
realizarCheckin	Se LIVRE ou RESERVADO → muda para OCUPADO e armazena/sobrescreve hospede.	check-in bem-sucedido	andar/apto inválido
realizarCheckout	Se OCUPADO → muda para LIVRE.	check-out bem-sucedido	andar/apto inválido
cancelarReserva	Se RESERVADO → muda para LIVRE.	cancelamento bem-sucedido	andar/apto inválido
mostrarMapa	Exibe matriz com . / R / 0.	(void)	(nunca)
consultarApartamento	Exibe status e, se não LIVRE, exibe dados do hóspede.	(void)	andar/apto inválido (lança exceção)
calcularTaxaOcupacao	Retorna (float) ocupados / total_aptos.	(float entre 0.0 e 1.0)	(nunca)
calcularTaxaReservas	Retorna (float) reservados / total_aptos.	(float entre 0.0 e 1.0)	(nunca)
cadastrarServico	Adiciona serviço à lista do hotel.	(void)	(nunca)
registrarConsumo	Adiciona consumo de serviço a um apto ocupado.	consumo registrado	andar/apto inválido
emitirFatura	Calcula total (diárias + consumos) e retorna Fatura.	(Fatura)	andar/apto inválido
HotelRepo.salvar HotelRepo.carregar	/ Serializa / desserializa o hotel em arquivo.	(void)	IOException

12

Menu do Sistema

Você deve implementar o menu na classe Main. Exemplo de estrutura:

```
package hotel.apresentacao;

import hotel.negocio.Hotel;
import hotel.modelo.Hospede;
import java.util.Scanner;

public class Main {
    private static Hotel hotel = new Hotel();
    private static Scanner scanner = new Scanner(System.in);

    private static Hospede lerHospede() {
        System.out.print("CPF: ");
        String cpf = scanner.nextLine();
        System.out.print("Nome: ");
        String nome = scanner.nextLine();
        System.out.print("Endereço: ");
        String endereco = scanner.nextLine();
        System.out.print("Celular: ");
        String celular = scanner.nextLine();
        System.out.print("Email: ");
        String email = scanner.nextLine();
        return new Hospede(cpf, nome, endereco, celular, email);
    }

    public static void main(String[] args) {
        int opcao;
        do {
            System.out.println("\n=== SISTEMA DE GESTAO DE HOTEL ===");
            System.out.println("1 - Reservar apartamento");
            System.out.println("2 - Realizar Check-in");
            System.out.println("3 - Realizar Check-out");
            System.out.println("4 - Cancelar reserva");
            System.out.println("5 - Mostrar mapa de ocupacao");
            System.out.println("6 - Consultar apartamento");
```

```

System.out.println("7 - Mostrar taxas de ocupacao e reservas");
System.out.println("8 - Cadastrar servico no cardapio");
System.out.println("9 - Registrar consumo de servico");
System.out.println("10 - Emitir fatura de apartamento");
System.out.println("11 - Relatorio de ocupacao");
System.out.println("12 - Ranking de servicos");
System.out.println("13 - Salvar dados em arquivo");
System.out.println("14 - Carregar dados de arquivo");
System.out.println("0 - Sair");
System.out.print("Opcao: ");
opcao = Integer.parseInt(scanner.nextLine());

switch (opcao) {
    case 1:
        // reservar
        break;
    case 2:
        // check-in
        break;
    // ... outros casos
}
} while (opcao != 0);
}

```

13

Validações Obrigatórias

Seu código **deve** validar:

- **Andar:** entre 0 e (NUM_ANDARES - 1) (inclusive)
- **Número do apartamento:** entre 0 e (APTOS_POR_ANDAR - 1) (inclusive)

Estratégia recomendada: Lance `IllegalArgumentException` para parâmetros inválidos nos métodos públicos da classe `Hotel`. O `Main` captura e exibe uma mensagem amigável.

14

Boas Práticas de Orientação a Objetos e Engenharia de Software

- **Encapsulamento:** Atributos `private`, acesso controlado.
- **Imutabilidade:** As classes `Hospede` e `Servico` devem ser imutáveis (sem *setters*).
- **Herança:** `ApartamentoSimples` e `ApartamentoPremium` herdam de `Apartamento`. Use `super()` para reaproveitar o construtor da base.
- **Composição:** `Consumo` associa `Servico` a `Apartamento`. `Fatura` agrega `Consumo`. Prefira composição sobre herança quando a relação é “tem-um”.
- **Responsabilidade única:** Cada classe tem um propósito claro.
 - `Hospede`: só dados do cliente.
 - `Apartamento`: estado de uma unidade + regras de transição.
 - `ApartamentoSimples/Premium`: preço da diária (herança).
 - `Servico`: item do cardápio.
 - `Consumo`: relação serviço-apartamento.
 - `Fatura`: cálculo de total.
 - `Hotel`: gerencia a coleção e coordena operações.
 - `HotelRepo`: persistência.
 - `RelatorioSvc`: geração de relatórios.
 - `Main`: interface com o usuário.
- **Evite repetição:** Crie métodos privados auxiliares quando necessário.
- **Use constantes:** `NUM_ANDARES`, `APTOS_POR_ANDAR`.
- **Trate exceções no lugar certo:** Valide nos métodos públicos do `Hotel`, capture no `Main`.

15

Entregáveis

1. **Código fonte completo** (todas as classes Java, organizadas no pacote hotel, com todos os módulos integrados).
2. **Arquivo de testes** (hotel/apresentacao/HotelTest.java) com todos os testes unitários e de integração, evidenciando o ciclo TDD no histórico git.
3. **Documentação Javadoc** em todos os métodos públicos de todas as classes.
4. **Relatório da equipe** (2-3 páginas) descrevendo:
 - A estrutura de classes e a responsabilidade de cada módulo.
 - Principais decisões de projeto (ex: por que herança vs composição em cada caso).
 - **Distribuição de tarefas** entre os membros e como a integração foi feita.
 - Dificuldades encontradas e como foram resolvidas.
 - Evidência de que o programa atende aos REQ01-REQ14.

16

Critérios de Avaliação

Os critérios são divididos em dois grupos conforme a natureza da avaliação:

16.1. CJ11 e CJ21: Avaliação TDD (demonstração em aula, 10 min por grupo)

Critério	Peso	O que significa
Corretude	50%	Testes passam? Código compila e atende os REQs da entrega.
TDD	30%	<i>Commits</i> seguem RED → GREEN → REFACTOR. Testes escritos antes da implementação.
Tratamento de exceções	20%	Testes de caso triste implementados e exceções lançadas para entradas inválidas.

16.2. CJ12A/B: Avaliação de Apresentação e Documentação Técnica (Operações + Hierarquia)

Critério	Peso	O que significa
Documentação	25%	Javadoc nos métodos públicos implementados. Relatório da equipe completo (2-3 páginas).
Operações básicas	20%	Hotel.java com reservar, check-in, check-out, cancelar, mapa e taxas.
Hierarquia	15%	ApartamentoSimples, ApartamentoPremium implementados e integrados.

16. Critérios de Avaliação

Integração entre classes	20%	Módulos A e B funcionam corretamente quando integrados.
Menu	10%	Main.java com opções para as operações implementadas.
Defesa oral	10%	Explicar o código, justificar decisões de modelagem, responder perguntas sobre o módulo próprio.

16.3. CJ22A/B: Avaliação de Apresentação e Documentação Técnica (Final)

Critério	Peso	O que significa
Documentação	25%	Javadoc em todos os métodos públicos de todas as classes. Relatório da equipe final.
Funcionalidades completas	20%	Todos os REQ01-REQ14 implementados e funcionando.
Persistência	15%	HotelRepo serializa e recupera o estado corretamente.
Relatórios gerenciais	15%	RelatorioSvc produz relatórios conforme REQ14.
Integração entre classes	15%	Módulos A-E funcionam corretamente quando integrados.
Defesa oral	10%	Explicar o código, justificar decisões de modelagem, responder perguntas sobre o módulo próprio.

17

Calendário de Entregas

As entregas são cumulativas — o código de cada etapa deve estar funcional para a etapa seguinte. Cada membro da equipe desenvolve seu módulo em paralelo, e a integração ocorre antes de cada entrega.

Entrega	Conteúdo	Avaliação
CJ11: Núcleo + Modelagem 17/09	Módulo A (core): Status.java, Hospede.java, Apartamento.java com transições de estado. De- finição dos contratos en- tre módulos.	TDD: demonstração em aula
CJ12A/B: Operações + Hierarquia 30/09 e 01/10	Módulo A: Hotel.java (operações bá- sicas). Módulo B: ApartamentoSimples, ApartamentoPremium. Inte- gração A+B.	Apresentação e docu- mentação técnica
CJ21: Serviços + Fatura 05/11	Módulo A: <i>check-in/out</i> completo, taxas. Módulo C: Servico, Consumo, Fatura. In- tegração A+C.	TDD: demonstração em aula
CJ22A/B: Final + Persistência +	Módulo D: HotelRepo. Módulo E:	Apresentação e docu- mentação técnica

17. Calendário de Entregas

Relatórios 12/11 e 18/11	RelatorioSvc. Main.java completo. Javadoc em todas as classes. Relatório da equipe.
-----------------------------	---

Distribuição Sugerida de Módulos

A alocação é flexível e deve se adaptar ao tamanho da equipe. Abaixo uma sugestão:

Módulo	Classes	Membros alocados
A — Núcleo (core)	Status, Hospede, Apartamento, Hotel, Main	1-2 alunos
B — Hierarquia	ApartamentoSimples, ApartamentoPremium	1 aluno
C — Serviços e Fatura	Servico, Consumo, Fatura	1 aluno
D — Persistência	HotelRepo	1 aluno (opcional)
E — Relatórios	RelatorioSvc	1 aluno (opcional)

Grupos de 5 alunos: todos os módulos (A-E) são alocados, com o módulo A recebendo 2 alunos.

Grupos de 4 alunos: o módulo A fica com 1 aluno, e D (Persistência) e E (Relatórios) são fundidos: um mesmo aluno implementa ambos. Alternativamente, o grupo pode redistribuir conforme suas habilidades.

Os módulos D e E são propositalmente menores e implementados apenas na reta final do projeto, o que torna viável um único aluno acumulá-los.

18

Dicas para Começar

1. **Definam os contratos entre módulos:** alinhem as assinaturas dos métodos que serão chamados entre módulos diferentes.
2. **Módulo A (core)** começa primeiro: Status, Hospede, Apartamento, Hotel com operações básicas.
3. **Módulo B (hierarquia)** pode começar em paralelo assim que Apartamento estiver pronto.
4. **Módulos C (serviços) e D (fatura)** dependem de Hotel e Apartamento: inicie após a entrega CJ11.
5. **Módulos E (persistência) e F (relatórios)** dependem de todos os anteriores: inicie após a entrega CJ21.
6. **Integrem continuamente:** usem git com *branches* e façam *merge* regularmente.
7. **Sigam TDD:** para cada método, primeiro o teste (RED), depois a implementação (GREEN), depois refatoração (REFACTOR). O git log deve mostrar esta sequência. Cada membro testa seu módulo isoladamente antes de integrar.
8. **Documentem tudo com Javadoc:** cada membro documenta seu módulo.
9. **Escrevam o relatório em equipe** destacando a contribuição individual de cada membro.
10. **Pratiquem a defesa oral:** cada membro deve saber explicar o código do seu módulo e como ele se integra aos demais.

19

Compilação e Execução

= **Compilar** (os .class vão para build/)
`javac -d build hotel/**/*.java`

= **Executar** (a partir de build/)
`java -cp build hotel.Main`

= **Gerar** documentação Javadoc
`javadoc -d doc hotel/**/*.java`

20

Observações Finais

- **Copiar código da internet ou de outros grupos é proibido.** A defesa oral será decisiva.
- **Uso de IA é permitido como ferramenta de apoio** (explicar conceitos, gerar esqueleto), mas cada membro deve ser capaz de explicar **cada linha do código do seu módulo** durante a defesa.
- **A integração é responsabilidade de toda a equipe:** não deixe para a última semana.
- **Não use variáveis públicas:** isso viola encapsulamento e resultará em perda de pontos.
- **Siga TDD:** escreva o teste **antes** do código. O arquivo `HotelTest.java` deve conter todos os testes da equipe. O `git log` será verificado como evidência do ciclo RED → GREEN → REFACTOR.

Bom trabalho! Lembre-se: sua equipe não é apenas um grupo de “programadores”. Vocês são **engenheiros de software** que projetam, implementam, testam, documentam e gerenciam configurações em equipe. Este projeto é a oportunidade de praticar **todos esses papéis** em um ambiente seguro e controlado.